DATA ANALYTICS

PYTHON DA ASSIGNMENT 1: DATA ANALYSIS USING NUMPY AND PANDAS

DATE OF SUBMISSION:25.10.2026

NAME: DURGADEVI V

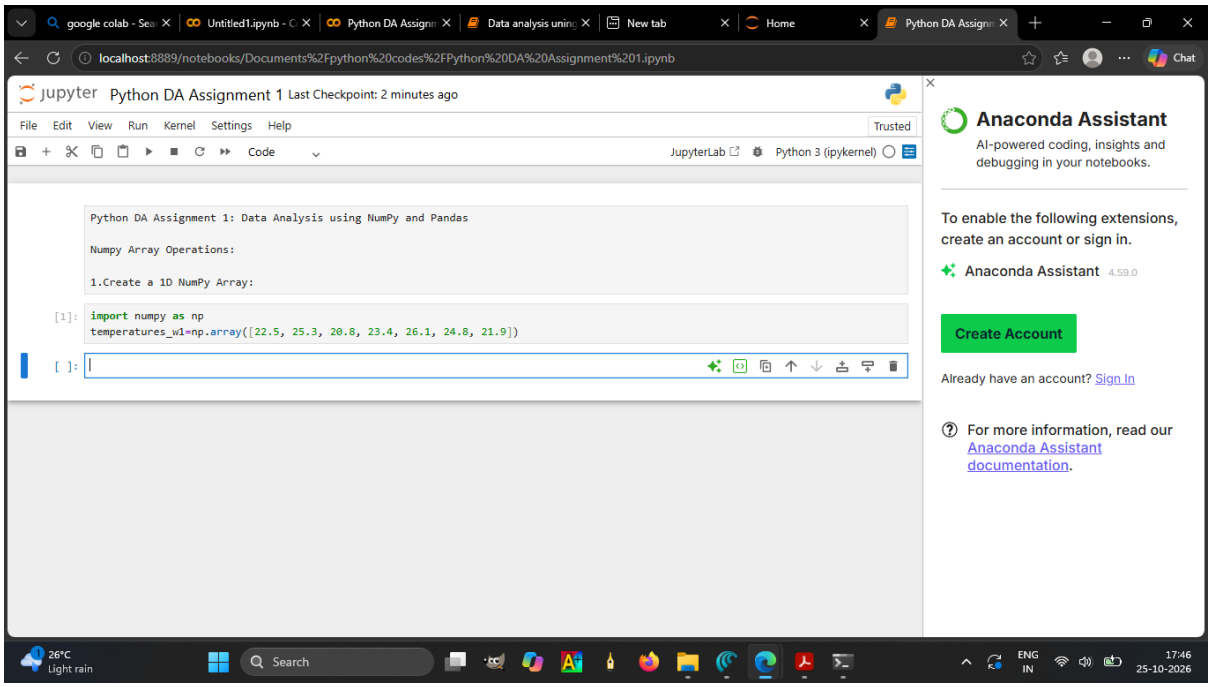
Data Analysis using NumPy and Pandas

NumPy Array Operations:

Task 1: Create a 1D NumPy Array

Solution :

Preview screenshot :



Explanation :

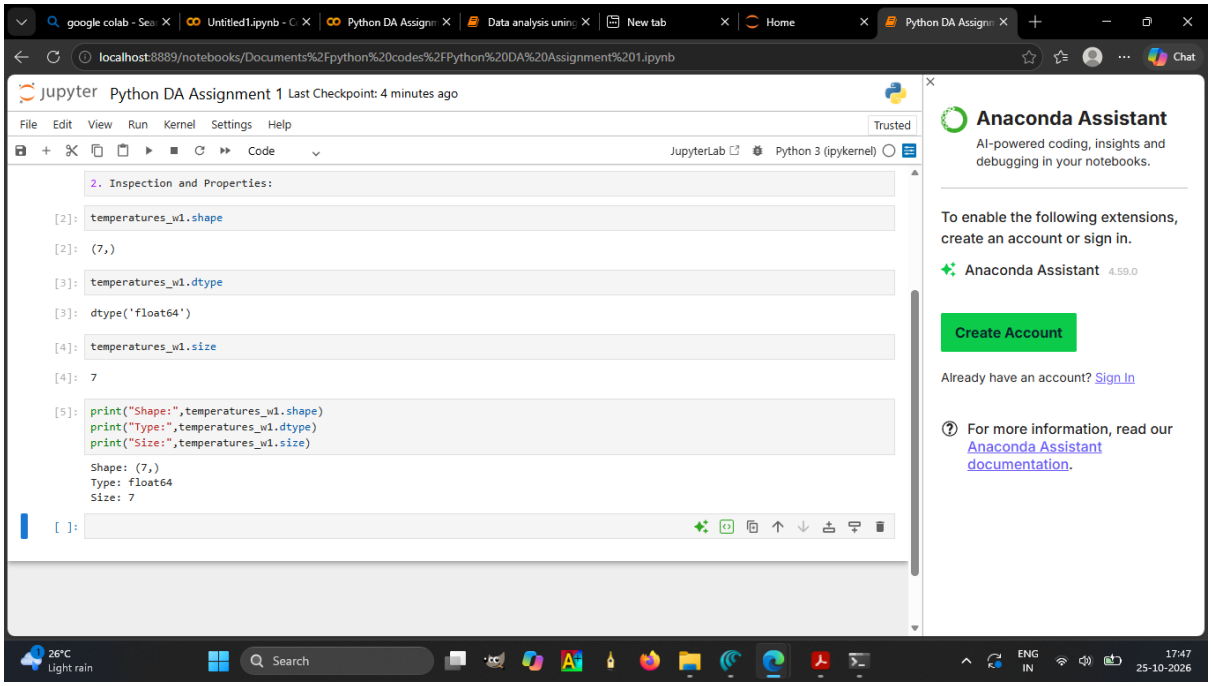
- First, I had to import NumPy by using code (import numpy as pd).

- And the create the 1D array named temperatures_w1 for 1 week had 7 degree Celsius and used this code for creating the 1D array.(temperatures_w1=np.array([22.5, 25.3, 20.8, 23.4, 26.1, 24.8, 21.9])
- The above mentioned screenshot represents the result.

Task 2: Inspection and Properties

Solution :

Preview screenshot :



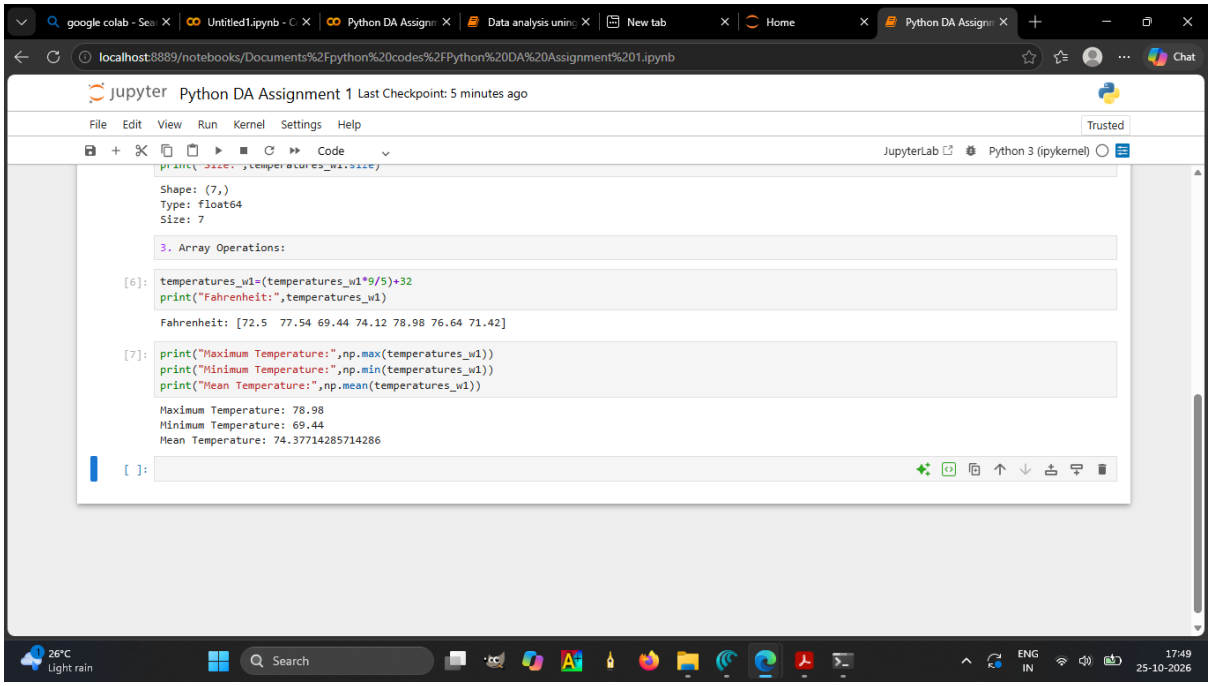
Explanation:

- For inspecting the 1D array I had to use the 'shape', 'dtype' and 'size' method. 'Shape' returns the shape of the 1D array ,'dtype' returns datatype and 'size' returns the number elements in the array.
- The above mentioned screenshot represents the result.

Task 3: Array Operations

Solution :

Preview screenshot :



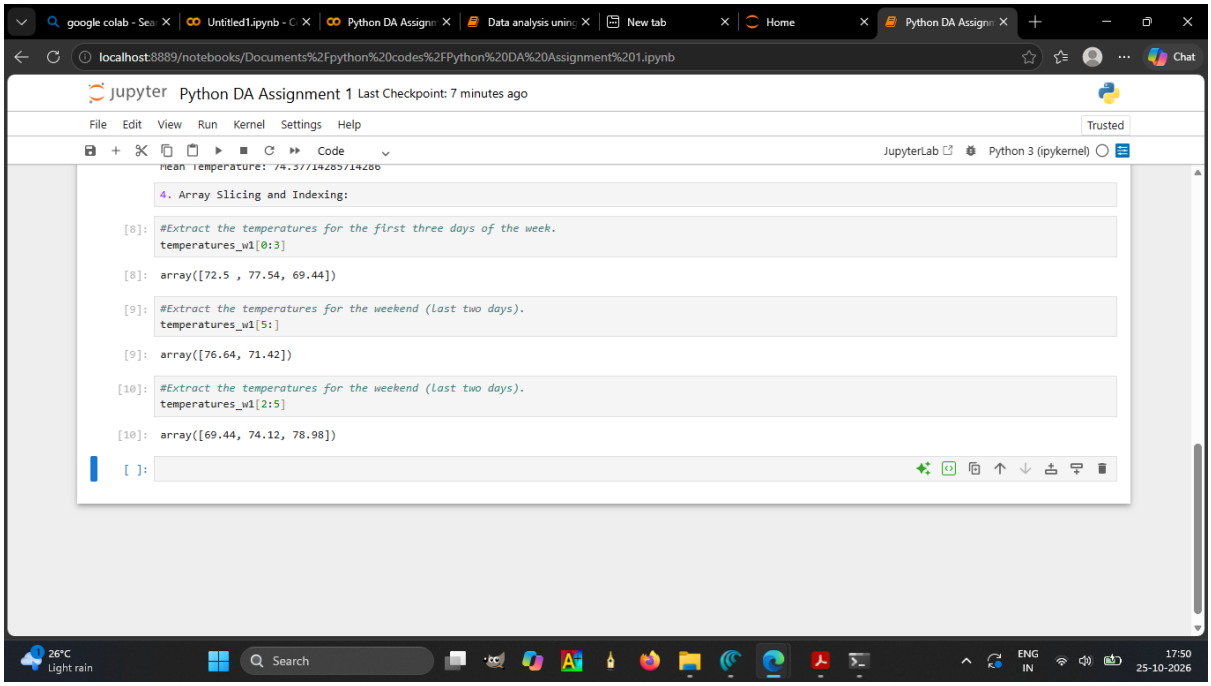
Explanation :

- To convert the temperatures_w1 from Celsius to Fahrenheit using this formula temperatures_w1=(temperatures_w1*9/5)+32.
- Then find the maximum temperature by using the max method (print("Maximum Temperature:",np.max(temperatures_w1))).
- Then find the minimum temperature by using the min method (print("Minimum Temperature:",np.min(temperatures_w1))).
- Then find the mean of temperature by using the mean method(print("Mean Temperature:",np.mean(temperatures_w1))).
- The above mentioned screenshot represents the result.

Task 4: Array Slicing and Indexing

Solution:

Preview screenshot :



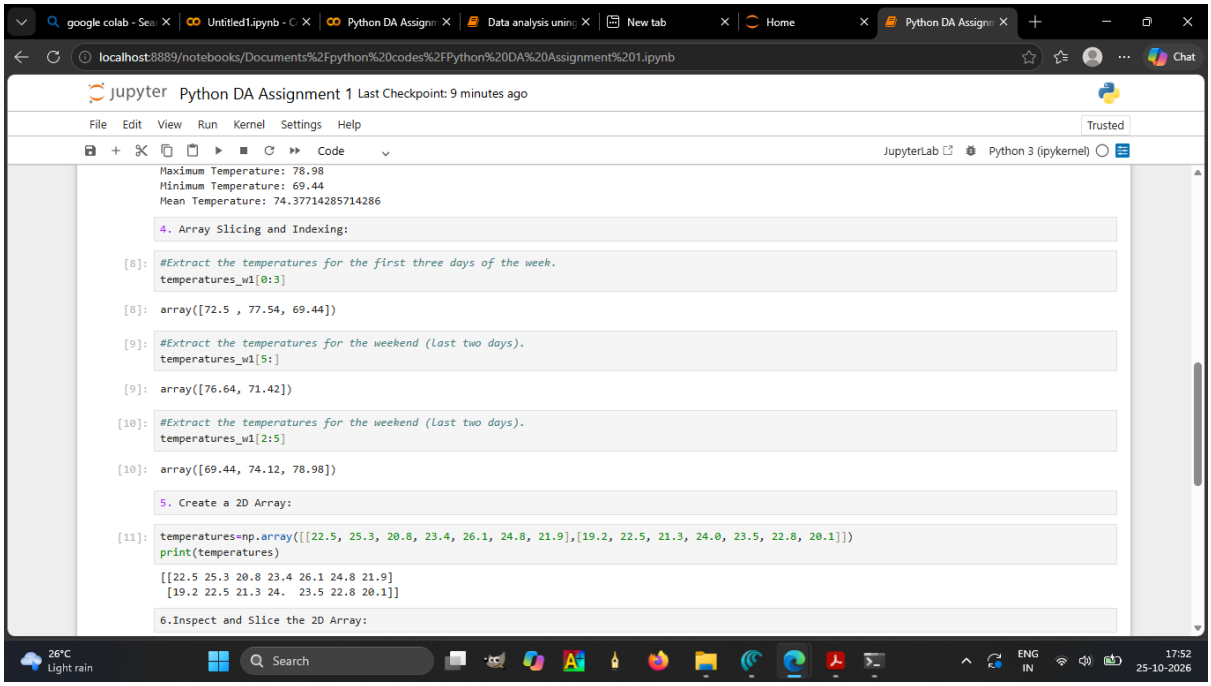
Explanation :

- For extract the temperatures for the first three days of the week, I had used this code temperatures_w1[0:3].
- And extract the temperatures for the weekend (last two days) ,I had used this code temperatures_w1[5:].
- And extract the temperatures for the weekend (last two days), I had used this code temperatures_w1[2:5].
- The above mentioned screenshot represents the result.

Task 5: Create a 2D Array

Solution :

Preview screenshot :



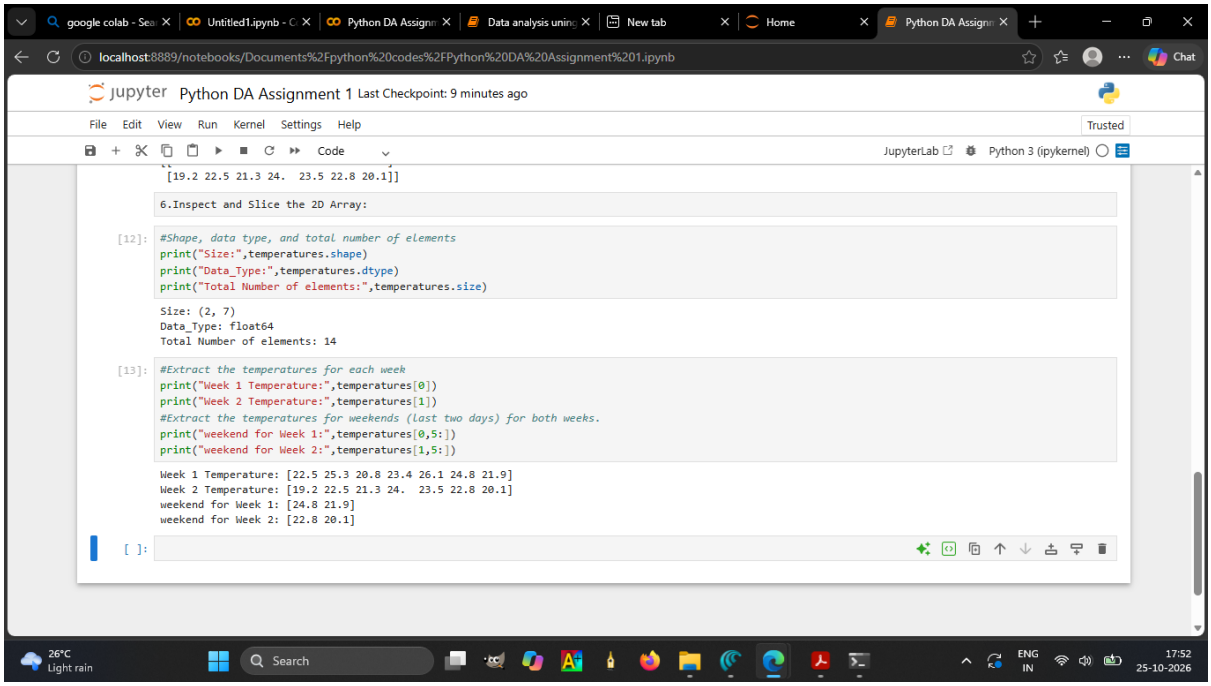
Explanation :

- To create the 2D array I had used this code temperatures=np.array([[22.5, 25.3, 20.8, 23.4, 26.1, 24.8, 21.9],[19.2, 22.5, 21.3, 24.0, 23.5, 22.8, 20.1]]).
- The above mentioned screenshot represents the result.

Task 6 : Inspect and Slice the 2D Array

Solution :

Preview screenshot :



Explanation :

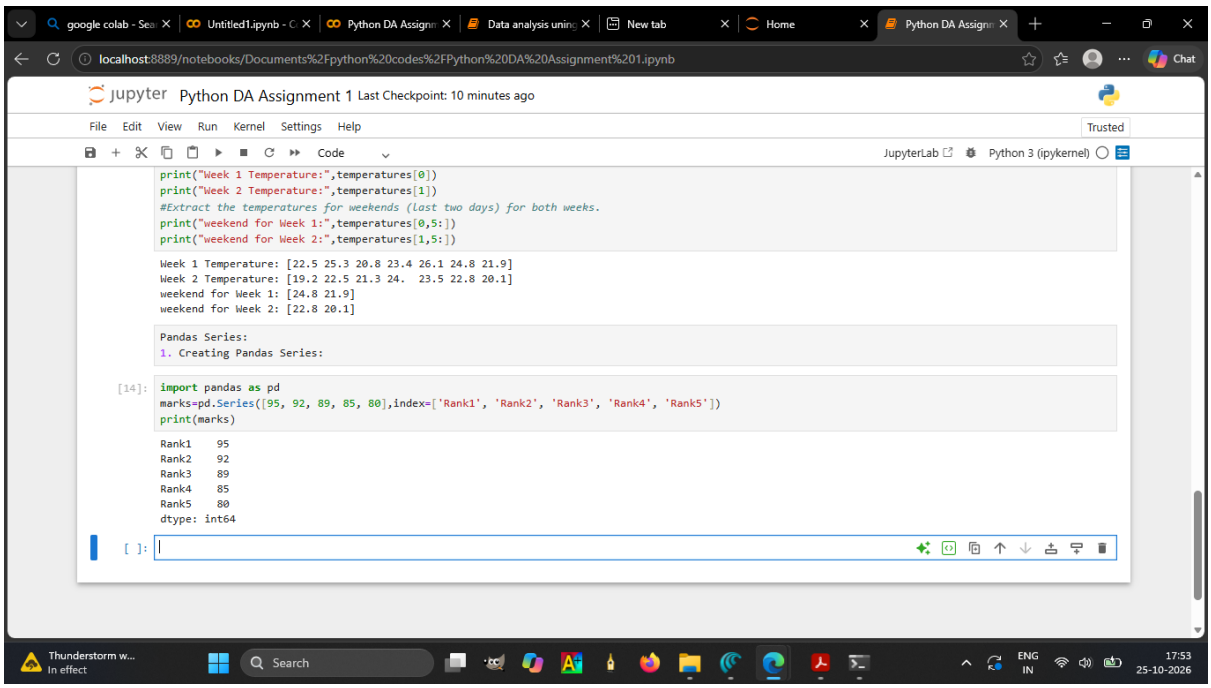
- For inspecting the 2D array I had to use the 'shape', 'dtype' and 'size' methods.
- Shape' returns the shape of the 2D array ,'dtype' returns datatype and 'size' returns the number elements in the 2D array.
- To extract the temperatures for each week, I had used the temperatures[0] and temperatures[1] indexing method .[0] returns the first row of the 2D array and [1] returns the second row of the 2D array.
- To extract the temperatures for weekends (last two days) for both weeks, I had used the temperatures[0,5:]). Here 0 represents the first row and 5: represents the element's starts from 5 and the last element so it returns the last two days of the first week and I had used the temperatures[1,5:]. Here 1 represents the first row(first week) and 5: represents the element's starts from 5 and the last element so it returns the last two days of the second week.
- The above mentioned screenshot represents the result.

Pandas Series:

Task 1: Creating Pandas Series

Solution :

Preview screenshot :



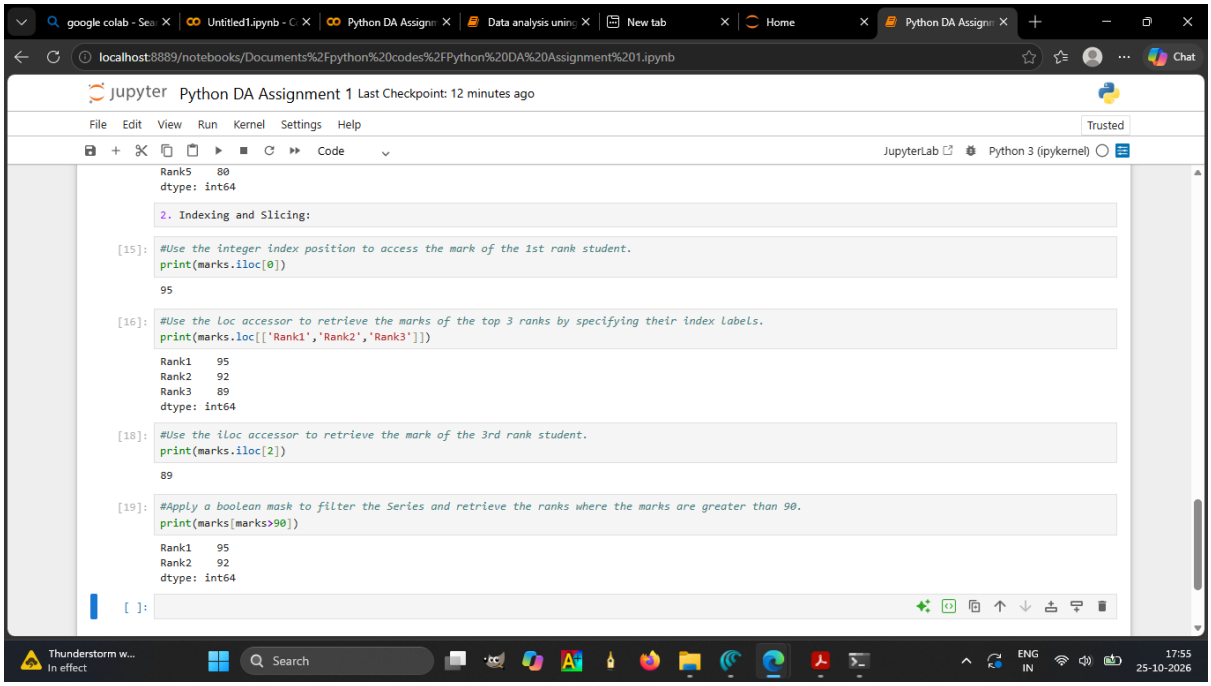
Explanation :

- For creating pandas series I had used the import method for importing pandas as pd.
- And then creating the series by using this code marks=pd.Series([95, 92, 89, 85, 80],index=['Rank1', 'Rank2', 'Rank3', 'Rank4', 'Rank5']).
- The above mentioned screenshot represents the result.

Task 2: Indexing and Slicing

Solution :

Preview screenshot :



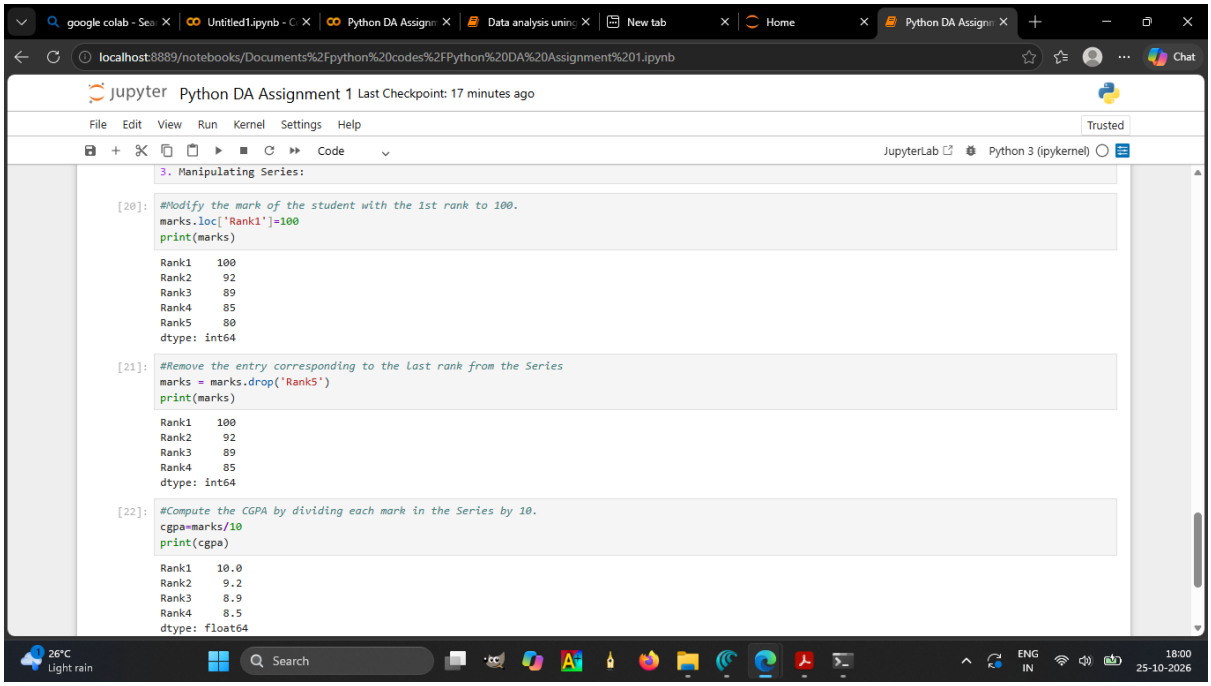
Explanation :

- I had used the integer index position to access the mark of the 1st rank student(print(marks[0])) and it returns the result as 95 but with a warning message -(/tmp/ipykernel_1436/3500771393.py:2: Future Warning: Series.__getitem__ treating keys as positions is deprecated. In a future version, integer keys will always be treated as labels (consistent with DataFrame behavior). To access a value by position, use `ser.iloc[pos]` print(marks[0])).
- So, the best method for return the 1st rank student is mark.iloc[0].
- Next, I had used the loc accessor to retrieve the marks of the top 3 ranks by specifying their index labels (print(marks.loc[['Rank1','Rank2','Rank3']])) and it's returns the rank1 ,rank2 rank 3 marks with indexing.
- Next, I had used the iloc accessor to retrieve the mark of the 3rd rank student(print(marks.iloc[2])) and it's retuns the 89 as result.
- Next, I had used the boolean mask to filter the Series and retrieve the ranks where the marks are greater than 90 (print(marks[marks>90])) and it's retuns the rank1 and rank2 marks,
- Above mention screenshots represents the result.

Task 3: Manipulating Series

Solution :

Preview screenshot :



Explanation:

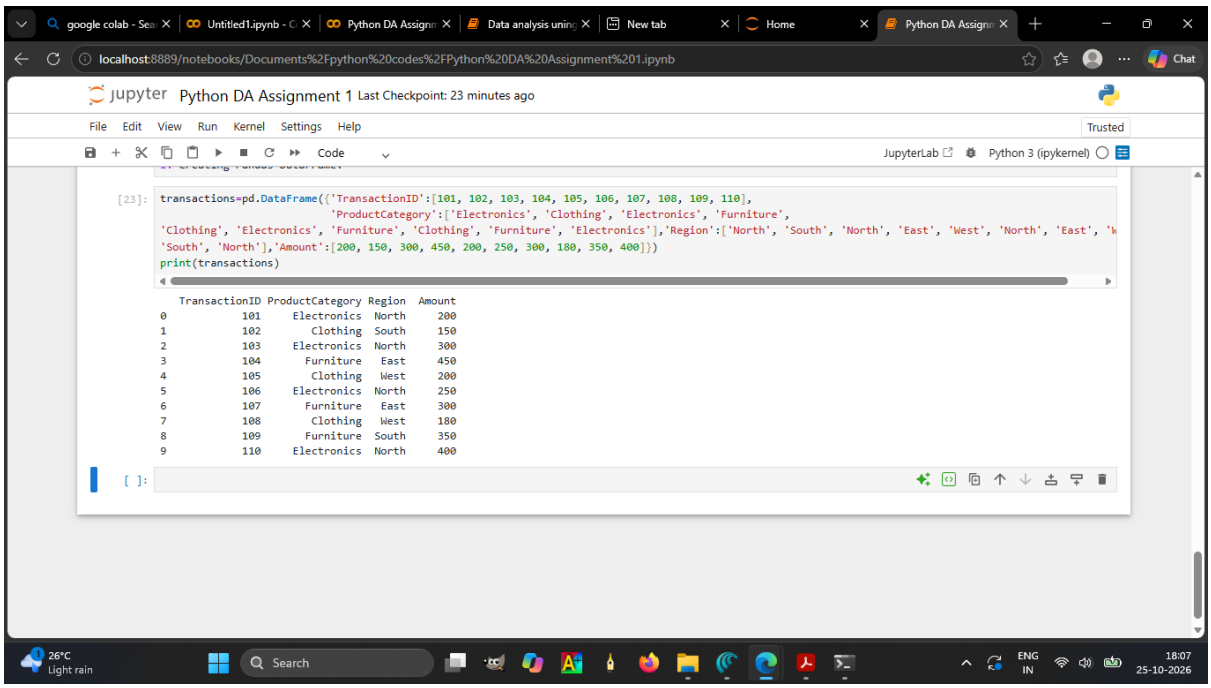
- For modify the mark of the student with the 1st rank to 100 I had used the(marks.loc['Rank1']=100) code .
- And for removing the entry corresponding to the last rank from the Series I used the(marks = marks.drop('Rank5')) code.
- And for computing the CGPA by dividing each mark in the Series by 10 I used the(cgpa=marks/10) code.
- Above mentioned screenshot represents the results.

Pandas Data Frame:

Task 1: Creating Pandas DataFrame

Solution :

Preview screenshot :



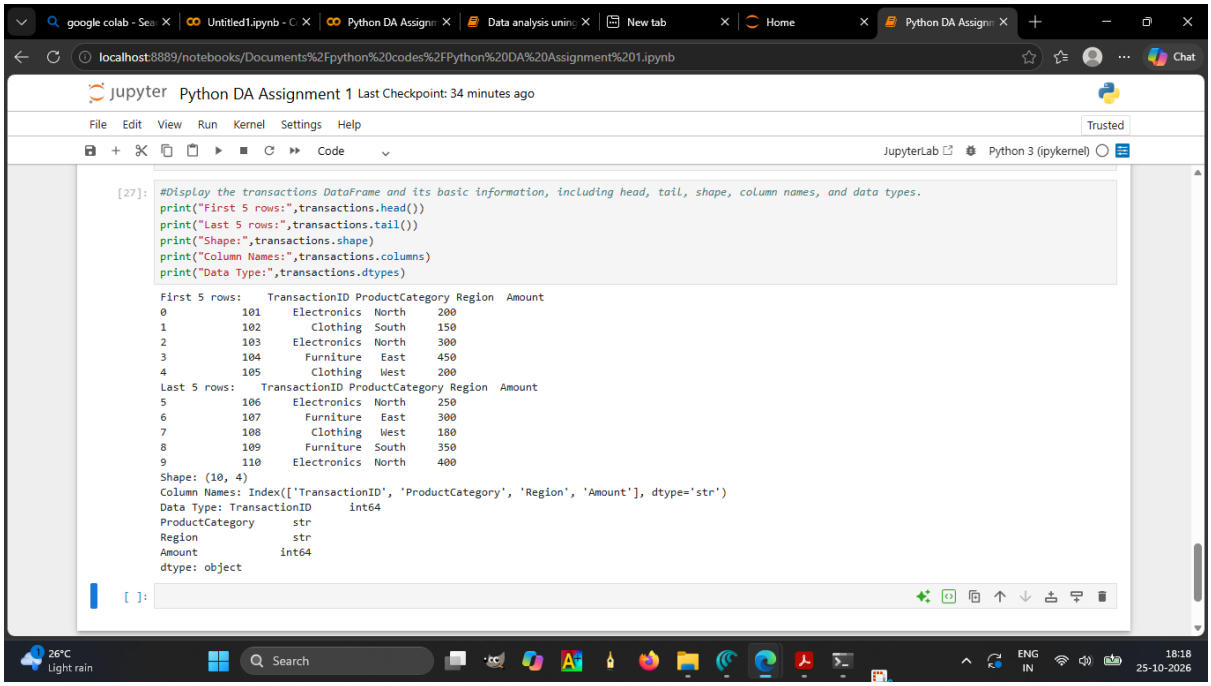
Explanation:

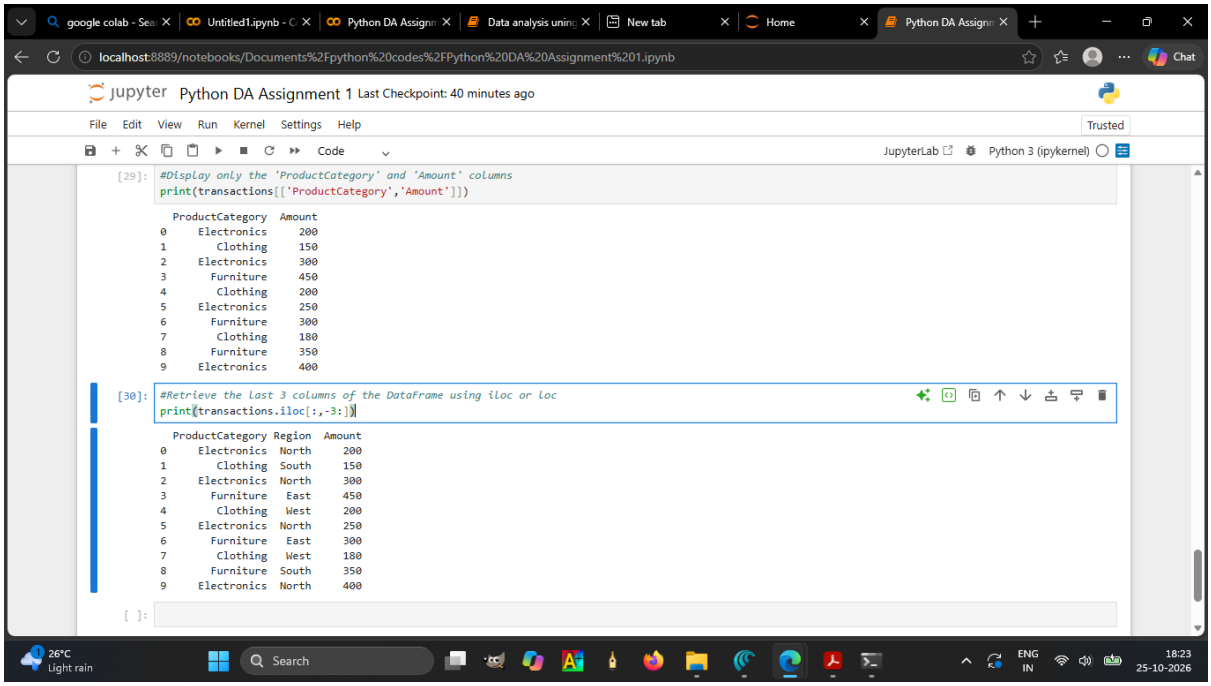
- I had created the data frame with given data.
- The above mentioned screenshot represents the result.

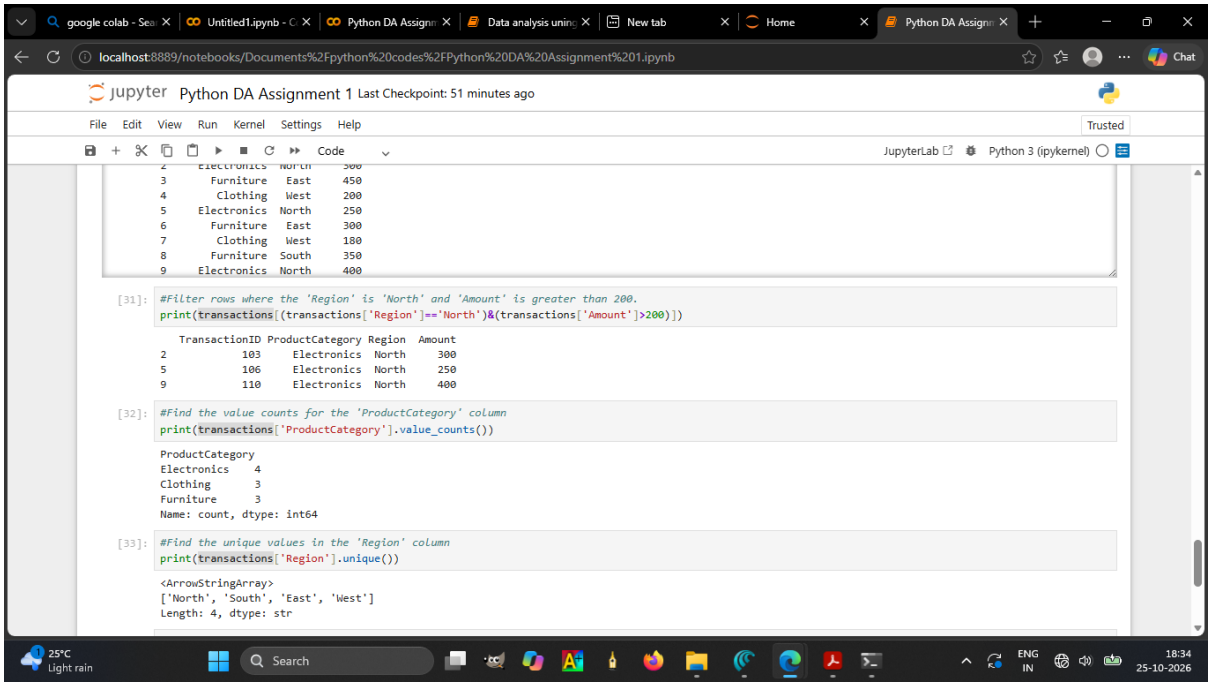
Task 2: Data Exploration

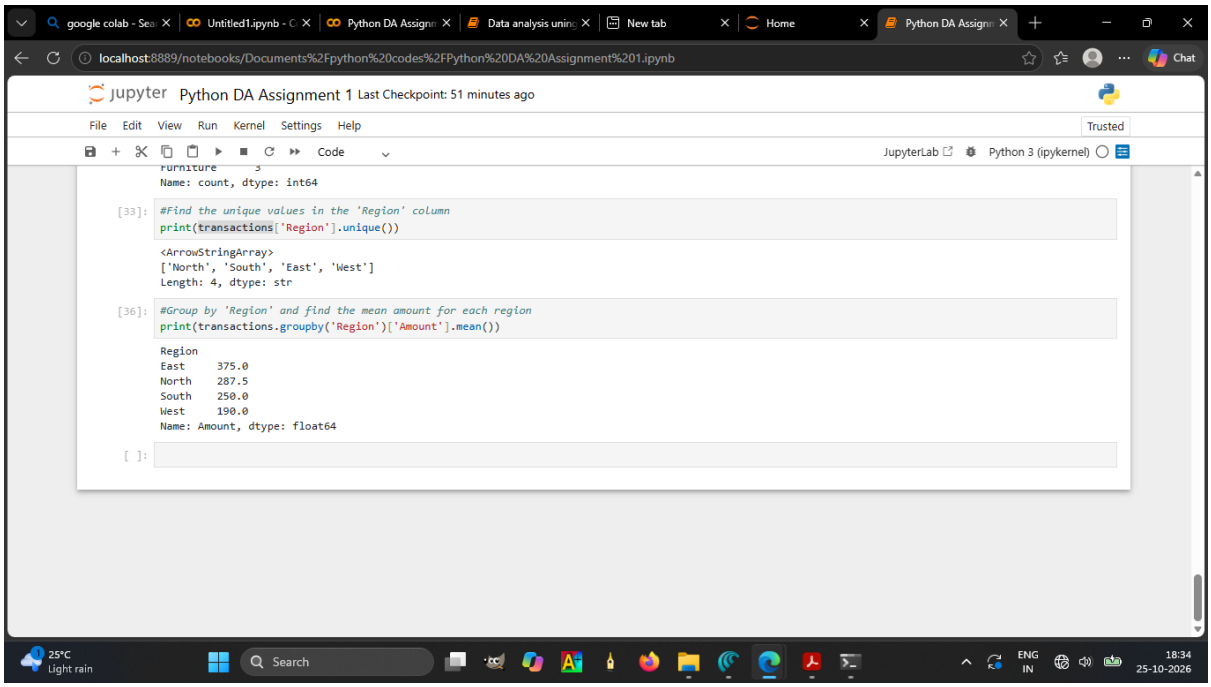
Solution :

Preview screenshot :









Explanation:

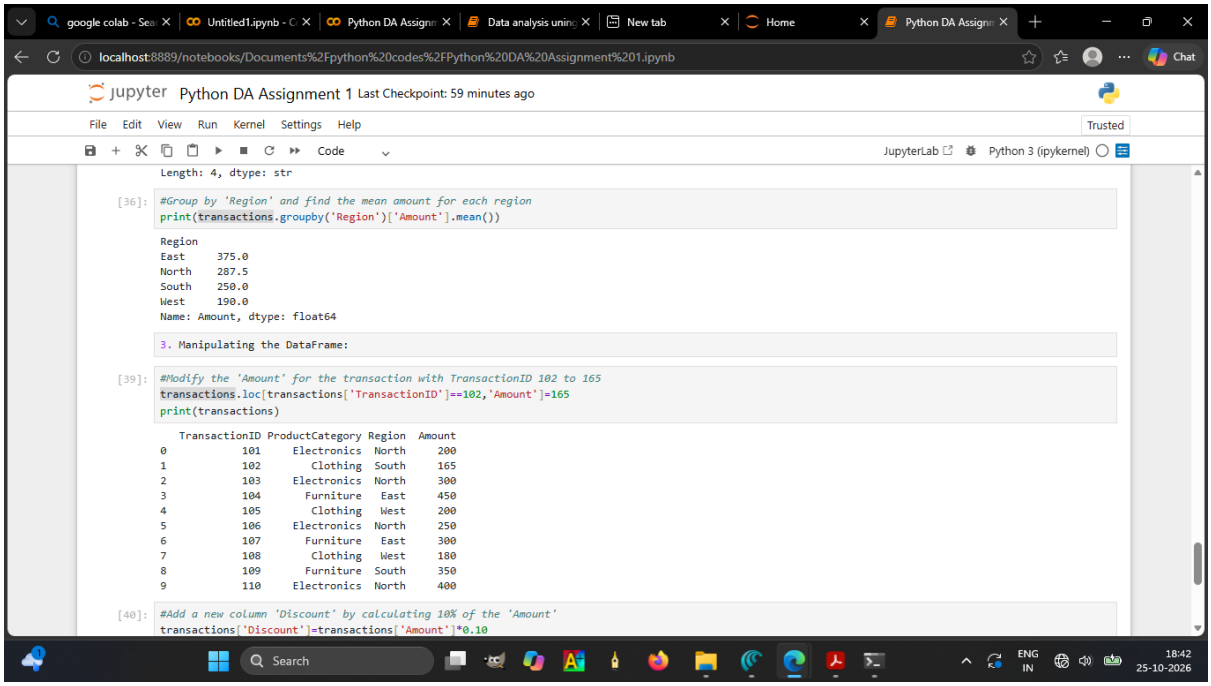
- I had used the head, tail, shape,columns,dtypes method for displaying the first 5 row, last 5 rows, shape of the data frame,column names of the data frame and datatype of the data frames.
- For display only the 'ProductCategory' and 'Amount' columns, I had used this code(print(transactions[['ProductCategory','Amount']]))

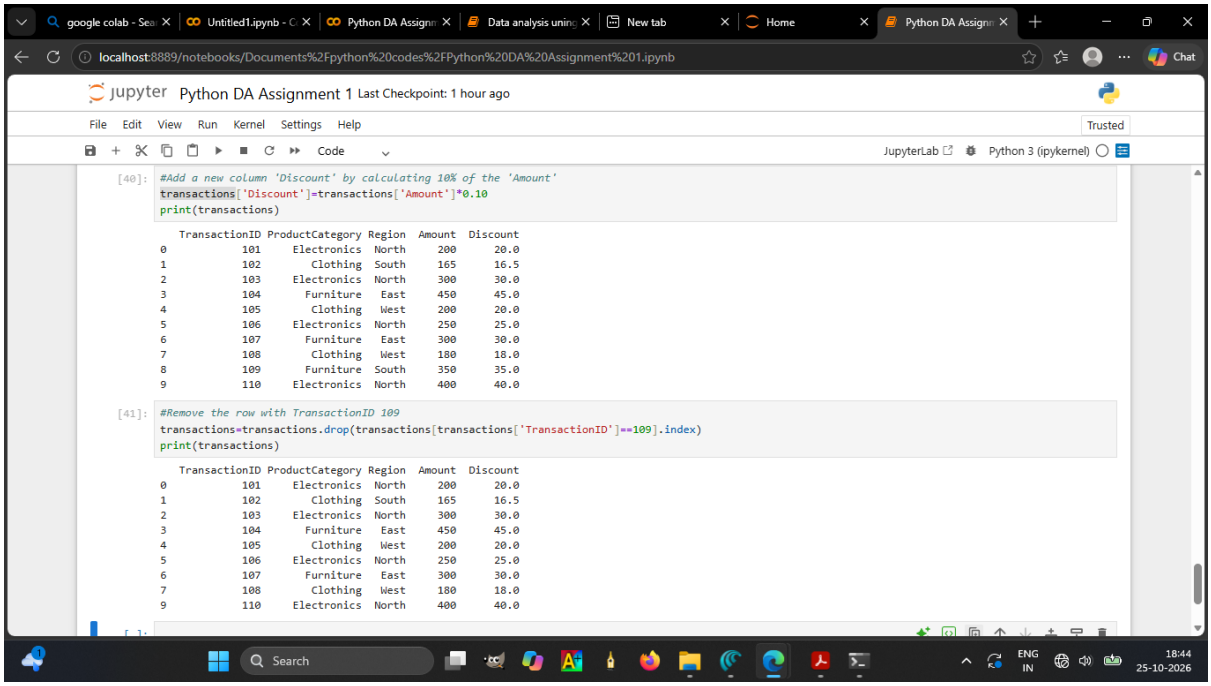
- For retrieve the last 3 columns of the DataFrame, I had used this code(print(transactions.iloc[:,-3:]))
- For filtering rows where the 'Region' is 'North' and 'Amount' is greater than 200, I had used this code(print(transactions[(transactions['Region']=='North')&(transactions['Amount']>200)])).
- To find the value counts for the 'ProductCategory' column, I had used this code (print(transactions['ProductCategory'].value_counts()))
- To find the unique values in the 'Region' column, I had used this code(print(transactions['Region'].unique()))
- For grouped by 'Region' and find the mean amount for each region , I had used this code (print(transactions.groupby('Region')['Amount'].mean()))
- The above mentioned screenshot represents the result.

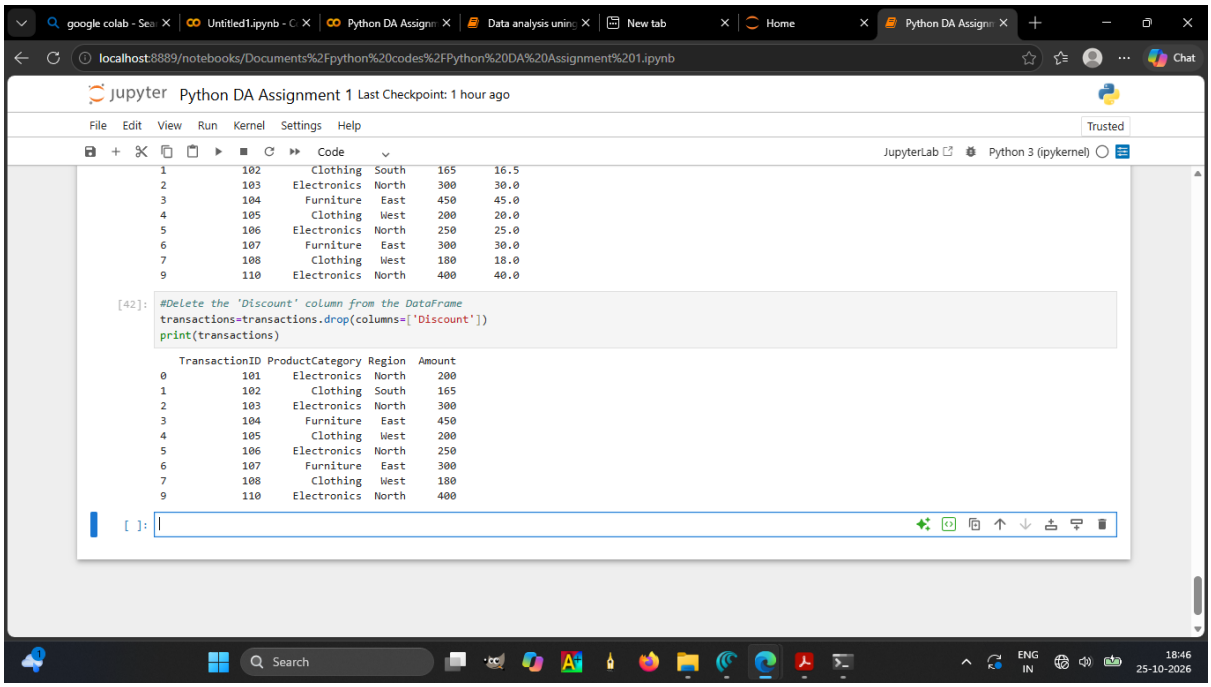
Task 3: Manipulating the Data Frame

Solution:

Preview screenshot :







Explanation:

- To modify the 'Amount' for the transaction with TransactionID 102 to 165loc indexing method and print the result.
- For adding a new column 'Discount' by calculating 10% of the 'Amount', I had used this code(transactions['Discount']=transactions['Amount']*0.10) and print the result.
- To remove the row with TransactionID 109, I had used the drop with index method.

- To delete the 'Discount' column from the Data Frame, I had used the used the drop with column method.
- Above mention screenshots represents the results.